



**FACULTY OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY DEPARTMENT OF COMMUNICATION TECHNOLOGY
AND NETWORK**

**GROUP PROJECT
DESIGN AND ANALYSIS OF ALGORITHMS (CSC4202)**

SEMESTER II 2025/2026

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSC 4202

GROUP: 2

PROJECT TITLE: LANDSLIDE EMERGENCY SUPPLY DELIVERY OPTIMIZATION

LECTURER: DR NUR ARZILAWATI MD YUNUS

NAME	MATRIC NUMBER
1. RASYIDAH LIANA BINTI ABD HALIM	222239
2. NUR SOFEA BINTI MUHALIS	224704
3. NURUL SYAZANA BINTI ZAIDI	225552
4. NURUL AINI BINTI ISMAIL	225068
5. ALYA MAISARAH HANNANI BINTI AHMAD SUKRI	226080

TABLE OF CONTENTS

1.0 Introduction.....	3
1.1 Background of the Disaster Scenario.....	3
1.2 Problem Statement.....	4
1.3 Project Objectives.....	5
1.4 Importance of Finding an Optimal Solution.....	5
2.0 Problem Definition and Scenario Development.....	6
2.1 Geographical Setting.....	6
2.2 Type of Disaster.....	7
2.3 Damage Impact.....	7
2.4 Decision-Making Requirements.....	8
2.5 Expected Output.....	8
2.6 Constraints.....	9
2.7 Illustration or Maps.....	10
3.0 Development of the Model.....	11
3.1 Data Types and Variables.....	11
3.2 Input and Output Definition.....	12
3.3 Objective Function.....	13
3.4 Assumptions and Constraints.....	13
3.5 Sample Input and Output.....	14
4.0 Algorithm Selection and Justification.....	16
4.1 Review of Sorting Algorithms.....	16
4.2 Review of Divide and Conquer.....	17
4.3 Review of Dynamic Programming.....	17
4.4 Review of Greedy Algorithms.....	18
4.5 Review of Graph Algorithms.....	19
4.6 Comparison Table.....	20
4.7 Selected Algorithm and Justification.....	20
Advantages in Emergency Evacuation.....	21
5.0 Algorithm Design.....	22
5.1 Algorithm Paradigm Overview.....	22
5.2 Pseudocode.....	22
5.3 Flowchart.....	23
5.4 Recurrence or Iterative Process.....	25
5.5 Optimization Function.....	26
6.0 Implementation.....	27
6.1 Development Environment.....	27
6.2 Java Program Structure.....	27
6.3 Important Code Snippets.....	27
6.4 Implementation Details.....	28

7.0 Program Testing and Results.....	28
7.1 Test Cases.....	28
7.2 Program Screenshots.....	29
7.3 Output Analysis.....	29
7.4 Discussion of Results.....	30
8.0 Algorithm Correctness and Complexity Analysis.....	31
8.1 Correctness Proof.....	31
8.2 Best-Case Time Complexity.....	31
8.3 Average-Case Time Complexity.....	32
8.4 Worst-Case Time Complexity.....	32
8.5 Asymptotic Analysis.....	32
9.0 Conclusion.....	33
10.0 References.....	35

1.0 Introduction

1.1 Background of the Disaster Scenario

Landslides represent one of the most destructive and recurring natural hazards in Malaysia (National Disaster Management Agency [NADMA], 2024). This phenomenon is heavily accelerated by the country's equatorial climate, which features intense, prolonged monsoon seasons, specifically the Northeast Monsoon (November to March) (Malaysian Meteorological Department, 2024). Geographically, the district of Bentong, located in the state of Pahang, is highly susceptible to slope failures. Situated along the foothills of the Titiwangsa Range and serving as a major transit corridor via the Karak Highway, Bentong's topography is characterized by steep slopes, heavily weathered residual soils, and dense river networks.

When exceptional rainfall saturates these hillside soils, pore-water pressure increases dramatically, reducing the shear strength of the earth and triggering sudden, catastrophic landslides (Rosly et al., 2023).

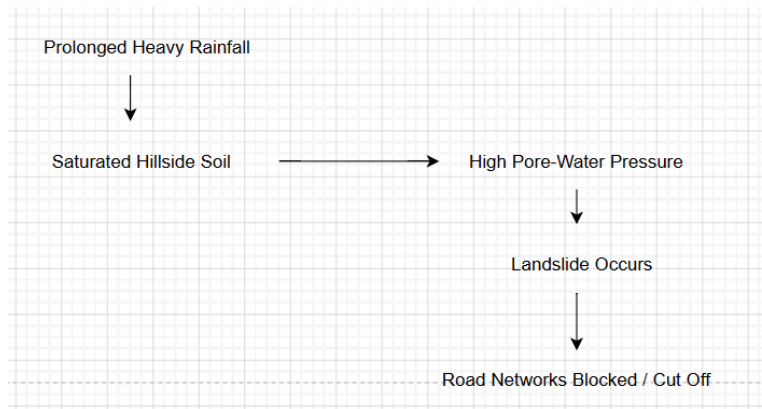


Figure 1: Causal Mechanism of Rainfall-Induced Landslides and Subsequent Network Disruption

The extensive interruption of important transportation networks is the secondary effect of these landslides, in addition to the direct structural damage. The logistics connections that connect urban centers to rural towns and indigenous (Orang Asli) villages deep within the countryside are sometimes severed by the burial of federal and state roadways under massive piles of debris.

Affected populations are abruptly cut off from electricity, clean drinking water, essential foods, and healthcare when these critical highways are closed.

In disaster response logistics, time is the ultimate constraint. The "Golden Hour" which is the critical window immediately following a disaster during which prompt medical intervention can drastically reduce mortality rates, demands instantaneous action (World Health Organisation, 2022).

However, it is risky and ineffective to send out emergency relief teams from the main Emergency Operations Center (EOC) without real-time spatial information. Using conventional GPS devices or manual route planning is ineffective because:

1. They do not dynamically account for real-time, multi-point road closures.
2. Human dispatchers cannot instantly calculate the mathematical permutations of alternative routes when multiple roads are blocked simultaneously.
3. They fail to optimize fuel, vehicle capacity, and rescue personnel allocation across multiple isolated villages (Geng, 2024).

An advanced computational strategy based on Algorithm Analysis and Design (AAD) is essential to reducing this. Rescue teams can use computer models that rapidly determine alternate routes by representing Bentong's actual road infrastructure as a mathematical network. This ensures that life-saving supplies avoid obstacles and reach vulnerable populations with the least amount of waiting time.

1.2 Problem Statement

Numerous landslides have happened simultaneously in the Bentong district as a result of persistent, intense rainfall, creating a challenging logistical situation (Malaysian Meteorological Department, 2024; National Disaster Management Agency [NADMA], 2024). Important transportation routes are blocked, making a number of major roads totally unusable and isolating a network of nearby towns. To deliver essential supplies to these remote areas, the local Emergency Operations Center (EOC) must send out a number of supply vans.

The core challenge lies in the dynamic and highly constrained nature of the post-disaster road network. The problem can be mathematically defined as navigating a disrupted graph $G = (V, E)$, where vertices (V) represent key intersections, villages, and the EOC, while edges (E) represent the connecting roads. Landslides have effectively altered this graph by introducing dynamic obstructions, rendering the weight (travel time/distance) of affected edges as infinite ($w(e) = \infty$).

Without an optimized, automated routing system, the emergency response suffers from several critical vulnerabilities:

- Suboptimal Routing Choices: Dispatching vehicles along unverified routes can lead to rescue teams encountering unexpected physical blockages, forcing tedious U-turns and backtracking that waste irreplaceable time and fuel.
- Resource Depletion: Inefficient pathing increases vehicle wear-and-tear and depletes limited fuel reserves in an area where gas stations may lack electrical power.
- Compounded Casualties: Delays in delivery mean that medical emergencies within the isolated villages remain unaddressed, directly resulting in preventable loss of life (United Nations Office for Disaster Risk Reduction [UNDRR], 2023) .

1.3 Project Objectives

To address the challenges identified in the problem statement, this project will achieve the following objectives:

- To Model the Post-Disaster Road Network: Construct a formal mathematical graph structure representing the geographical landscape of Bentong, mapping the Emergency Operations Centre, intersections, and isolated villages as vertices, and active/blocked roads as weighted edges.
- To Evaluate Algorithmic Paradigms: Conduct a rigorous theoretical review of various algorithm design techniques (including Sorting, Divide and Conquer, Dynamic Programming, Greedy approaches, and Graph-specific algorithms) to evaluate their strengths and architectural suitability for this specific optimization problem.
- To Design an Optimal Routing Algorithm: Architect a definitive, obstacle-avoiding shortest path algorithm tailored to handle the dynamic constraints of a post-landslide environment, ensuring complete correctness and efficiency.
- To Analyze Algorithmic Complexity: Formally evaluate the designed algorithm's correctness through mathematical properties and derive its asymptotic time complexity (Best, Average, and Worst-case scenarios) using Big-O (O) notation to guarantee high performance under data scaling.
- To Implement a Java-Based Decision Support Tool: Develop and compile a fully functional, production-ready Java application equipped with a clear user interface that enables disaster management personnel to input current blockages and instantly receive the optimal, high-efficiency routing path.
- To Document and Present the Engineering Solution: Compile the entire lifecycle of the algorithm's design, codebase, and experimental outputs into an open-access online portfolio (GitHub) and deliver an analytical technical presentation.

1.4 Importance of Finding an Optimal Solution

Finding the best route in traditional commercial logistics is motivated by economic KPIs, such as cutting delivery times and corporate operating expenses. Finding the best answer in disaster management, however, is important for human existence. The use of arbitrary or "good enough" heuristic solutions can have deadly results, because there is no margin for mistake.

THE DISASTER RESPONER CYCLE	
Suboptimal Solution	Optimal AAD Solution
• Guesswork & Backtracking • Wasted Fuel & Vehicles • Bottleneck at EOC	• Instant Obstacle Bypass • Maximized Fleet Lifespan • Synchronized Deliveries
Compound Casualties	Lives Saved ("Golden Hour")

Figure 2: Comparative Matrix of Suboptimal Manual Routing vs Mathematically Optimal AAD Solutions in Disaster Response

The strict importance of a mathematically optimal solution involves several critical variables:

- The "Golden Hour" of Triage: For individuals suffering from severe physical trauma due to structural collapses or landslides, survival rates drop exponentially every hour medical supplies are delayed. An optimal algorithm ensures that medical transport utilizes the absolute fastest path down to the exact minute (World Health Organization [WHO], 2023).
- Severe Resource Scarcity: Critical resources, such as operating off-road trucks, diesel fuel reserves, clean drinking water bottles, and qualified paramedics, are strictly limited during a localized emergency. The long-term sustainability of the rescue operation is severely hampered by inefficient routing, which causes empty miles, early vehicle problems, and fuel exhaustion.
- Mitigation of Cognitive Overload: Due to an overwhelming amount of chaotic, unstructured material from field investigation reports, disaster commanders work under extreme psychological strain. Human error is eliminated by employing a strict, deterministic algorithm created using sound AAD concepts. Decision-makers receive clear, actionable path commands from the system, which also immediately processes thousands of edge variations and filters out noise.
- Scalability for Widespread Chaos: If the landslide crisis expands to cover adjacent regions (such as Raub or Temerloh), manual coordination breaks down entirely. An algorithm proven to have a low asymptotic growth rate (e.g., $\mathcal{O}(V^2)$ or $\mathcal{O}(E + V \log V)$) guarantees that even as the inputs scale up to hundreds of vertices and thousands of damaged edges, the computer system will still return optimal paths in milliseconds, preventing catastrophic software lock-ups during a crisis.

2.0 Problem Definition and Scenario Development

2.1 Geographical Setting

The project is situated within the Bentong District, an administrative zone in western Pahang, Malaysia. Geographically, Bentong acts as the immediate gateway to the East Coast of Peninsular Malaysia, flanking the rugged terrains of the Main Range (*Banjaran Titiwangsa*) (Bentong Municipal Council, 2025). The district is highly defined by its undulating topography, steep valley walls, and microclimates that trap dense moisture, leading to severe localized precipitation.

The rural infrastructure is made up of narrow, twisting, unpaved state and village roads (Jalan Kampung) that interact with the main highway network (such as the Karak Highway and the former federal route FT68). These routes connect Orang Asli (indigenous) communities and

remote villages to the urban core by winding through thick jungles and rubber or oil palm plantations (Asnawi et al., 2024).

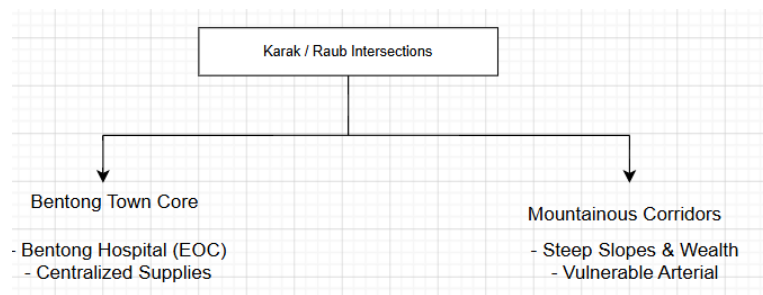


Figure 3 : Structural Hierarchy of the Bentong District Road Network Logistics and Supply Corridors.

For this disaster model, the localized Emergency Operations Centre (EOC) is established at Bentong Hospital. Located structurally in the urban core of Bentong town, this medical facility serves as the centralized logistics hub where medical teams, food rations, clean water filtration units, and heavy rescue vehicles are organized.

The surrounding network consists of vulnerable isolated villagers that depend on particular, low-redundancy valley roads for existence. These villages are located deep into the mountainous corridors of Janda Baik, Chamang, and Kampung Karak.

2.2 Type of Disaster

The catalytic disaster under study is a series of heavy rainfall-induced landslides, a catastrophic geological event triggered by the destabilization of slope materials. During the peak of the monsoon, Bentong can experience upwards of 150 mm to 300 mm of downpour within a single 24-hour cycle (Malaysian Meteorological Department, 2024).

The mechanics of this disaster involve two primary computational and physical impacts:

- The Saturation Threshold: The continuous rain infiltrates the residual granitic soil layers common to Pahang's hillsides, increasing the groundwater table and generating high pore-water pressure.
- Slope Failure and Network Corruption: Once the structural shear strength of the slope falls below the gravitational shear stress, thousands of tons of earth, boulders, and uprooted trees slide downward. In terms of network topology, these events instantly transform navigable, low-weight edges into completely impassable barriers (Rosly et al., 2023).

2.3 Damage Impact

These concurrent landslides have a cascade effect that goes well beyond actual road closures, making the situation extremely unstable for rescue crews:

- Complete Network Disconnection (Isolating Cut-off): Landslides along single-access valley roads create critical points of failure, physically trapping thousands of residents.
- Rapid Resource Depletion: Isolated villages immediately lose access to clean water pipelines and electrical grids. Without incoming emergency relief, local food stocks and critical medical supplies (such as insulin or basic trauma care items) deplete within hours.
- Exponentially Compounded Response Latency: Traditional rescue routes are rendered useless. Forcing vehicles onto unverified paths creates severe bottlenecks, forcing massive physical backtracking and wasting limited daylight hours.
- Dynamic, Multi-wave Closures: The landscape remains highly unstable post-disaster. Subsequent minor tremors or continued drizzle can trigger secondary landslides, meaning a route verified as "open" at 08:00 AM could become entirely blocked by 10:00 AM ((National Disaster Management Agency [NADMA], 2024).

2.4 Decision-Making Requirements

Disaster leaders cannot rely solely on intuition to function effectively in this logistical chaos. They need a high-fidelity, automated deterministic computing system that can meet the following algorithmic requirements:

- Single-Source Shortest Path (SSSP) Extraction: The routing engine must treat the EOC (Bentong Hospital) as the definitive source node (s) and calculate the absolute minimum cumulative weight (travel time) to every accessible destination node.
- Dynamic Edge Pruning: The system must actively accept live field reports regarding landslips and prune blocked edges from the working data structure, ensuring the pathing engine completely ignores unsafe roads.
- Accumulated Latency Cost Estimation: For every valid route discovered, the algorithm must sum the exact edge weights to output a precise, predictable travel duration in minutes, optimizing fleet dispatch schedules.
- Definitive Unreachability Flagging: If an isolated village's connecting edges are entirely severed ($w = \infty$), the system must instantly flag that node as Inaccessible. This informs decision-makers to bypass ground deployment and immediately escalate to air-drop operations (via helicopters) (National Disaster Management Agency [NADMA], 2024; United Nations Office for Disaster Risk Reduction [UNDRR], 2023).

2.5 Expected Output

In order to provide dispatch drivers with highly understandable, actionable instructions, the software engine must parse the network data. The sequential path traversal and the cost matrix must be explicitly provided in the output.

Class Output Profile Specification:

```
=====
BENTONG DISASTER RESPONSE ROUTING ENGINE - OUTPUT LOG
=====
[SOURCE NODE]: Bentong Hospital (EOC)

STATUS: SUCCESSFUL PATH COMPUTATION
-----
▶ Village 1: Bentong Hospital → Junction B → Village 1
  Total Estimated Travel Time: 18 Minutes
  Route Status                : CLEAR / OPTIMAL

▶ Village 2: Bentong Hospital → Junction B → Village 2
  Total Estimated Travel Time: 15 Minutes
  Route Status                : CLEAR / ALTERNATIVE DETOUR

▶ Village 3: No available route.
  Total Estimated Travel Time: INF (∞)
  Route Status                : CRITICAL CRASH - VILLAGE IS ISOLATED
  Action Required             : ALERT AIR DISPATCH (HELICOPTER REQ)
=====
```

Figure 4: System Output Profile Specification Template for Post-Disaster Routing Diagnostics

2.6 Constraints

The method must function within the following logical bounds to guarantee that the mathematical abstraction is exactly in line with actual physics:

- **Non-Negative Edge Weights:** For any active edge $e \in E$, the travel time weight $w(e)$ must strictly satisfy $w(e) > 0$. Time cannot be negative or zero ($w(e) \neq 0$), ensuring structural protection against infinite negative cycles during path relaxation.
- **Infinite Weight Mapping for Blockages:** Any road segment impacted by a landslide must have its weight adjusted to infinity ($w(e) = \infty$), or the edge must be fully purged from the adjacency list data structure.
- **Strict Rooted Directed/Undirected Traversal:** All transport manifests must originate strictly from the designated source vertex ($v_0 = \text{Bentong Hospital}$).
- **Temporal Volatility (Dynamic Graph Re-computation):** As road states change from open to blocked, the algorithm must be efficient enough to re-execute instantly without software latency, satisfying real-time constraints.
- **Strict Algorithmic Execution Bound:** The time complexity must be strictly sub-exponential which ideally is bounded within polynomial or log-linear limits, guaranteeing calculation returns within milliseconds even if input nodes scale up.

2.7 Illustration or Maps

To map this scenario into a concrete discrete structure, we model the Bentong regional landscape using a weighted, undirected graph $G = (V, E)$.

Graph Network Specification:

- Vertices (V): Total of 8 core operational nodes.
 - V_0 : Bentong Hospital (EOC) — *Source Node*
 - V_1, V_2, V_3 : Strategic Intersections — *Junction A, Junction B, Junction C*
 - V_4, V_5, V_6, V_7 : Distressed Target Areas — *Village 1, Village 2, Village 3, Village 4*
- Edges (E): The connecting road links, embedded with weights representing standard travel time in minutes.

Topology Layout and Landslide Impact Map:

Below is the structural map of the network layout. In this scenario, two severe landslides have occurred: Road (Junction B $\leftrightarrow$ Village 3) and Road (Junction C $\leftrightarrow$ Village 3) are completely blocked by debris, effectively severing Village 3 from the physical network grid.

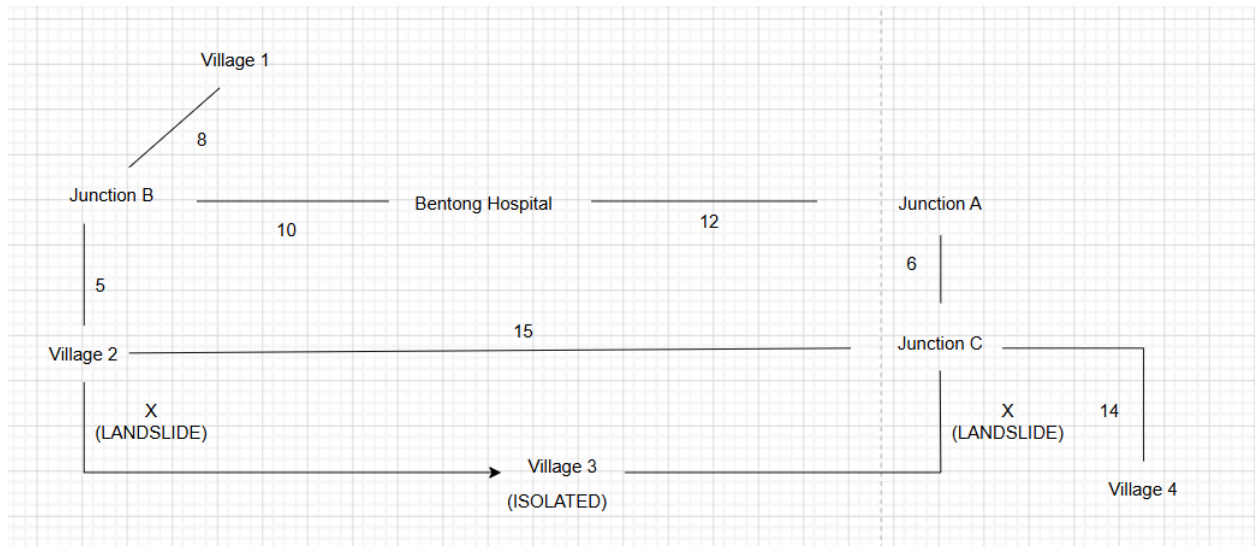


Figure 5: Weighted Graph Representation $G = (V, E)$ of Bentong's Road Network Showing Active Landslide Blockages and Node Isolation.

Complete Weight Matrix & State Table:

The underlying data structures will maintain edge configurations as shown in the operational lookup table below:

From (Vertex u)	To (Vertex v)	Default Weight	Current State	Active Weight in
--------------------	------------------	----------------	---------------	------------------

		(Mins)		System
Bentong Hospital	Junction B	10	Open	10
Bentong Hospital	Junction A	12	Open	12
Junction B	Village 1	8	Open	8
Junction B	Village 2	5	Open	5
Junction A	Junction C	6	Open	6
Junction C	Village 2	15	Open	15
Junction B	Village 3	9	Blocked	∞ (Purged)
Junction C	Village 3	11	Blocked	∞ (Purged)
Junction C	Village 4	14	Open	14

3.0 Development of the Model

This section explains how the real-world landslide emergency scenario is transformed into a computational graph model. The Bentong district road network is represented mathematically as weighted graph $G=(V,E)$, where vertices represent key locations such as the Emergency Operations Centre (EOC), junctions, and villages, while edges represent connecting roads with associated travel times.

3.1 Data Types and Variables

To implement the computational model, several data structures and variables are required to represent the road network and support the execution of Dijkstra's Algorithm. The road network is modelled as a weighted graph, where vertices represent important locations and edges represent road connections between locations. The main data types and variables used in the model are described as follows:

- Graph $G(V,E)$**
 Represents the road network, where **V** is the set of vertices (locations) and **E** is the set of edges (road segments). Each edge connects two vertices and contains an associated travel cost.
- Weight, $w(u,v)$**
 Represents the travel time required to move from vertex **u** to vertex **v**. Roads reported as blocked by landslides are assigned an infinite weight (∞) to prevent them from being selected during route computation.

- **distance[]**
An array used to store the shortest known distance from the source node to every vertex in the network. All values are initially set to infinity (∞), except for the source node, which is assigned a value of 0.
- **parent[]**
An array used to store the predecessor of each vertex in the shortest path. This information is used to reconstruct the final route after the algorithm has completed. All entries are initially assigned the value NIL.
- **visited[]**
A Boolean array used to indicate whether a vertex has been permanently processed by the algorithm. A value of TRUE indicates that the shortest distance to the vertex has been finalized, while FALSE indicates that the vertex has not yet been processed.
- **Priority Queue (Q)**
A priority queue is used to efficiently select the unvisited vertex with the smallest tentative distance during each iteration of the algorithm. This improves the efficiency of shortest-path computation.
- **Source Vertex (V_0)**
Represents the starting location of the route calculation. In this study, Bentong Hospital serves as the source vertex because it functions as the Emergency Operations Centre (EOC).
- **Route Status Flag**
A variable used to classify the accessibility of the computed route. The possible values are:
 - **CLEAR** – A normal route exists without encountering blocked roads.
 - **DETOUR** – An alternative route is required due to blocked road segments.
 - **INACCESSIBLE** – No feasible route exists between the source and destination.

3.2 Input and Output Definition

The system operates based on structured inputs derived from the disaster environment.

The input to the system consists of:

- The total number of vertices representing locations such as hospitals, junctions, and villages
- A list of weighted edges representing road connections and travel times
- The source node, which is the Emergency Operations Centre located at Bentong Hospital
- A set of blocked edges representing roads affected by landslides

The output generated by the system includes:

- The shortest travel time from the source node to each reachable village

- The optimal path taken to reach each destination node
- An indication of “INF” which means unreachable for villages that are completely isolated due to road blockages

This output provides emergency response teams with clear and actionable routing instructions for supply delivery.

3.3 Objective Function

The objective is to find a path $P = (v_0, v_1, \dots, v_k)$ from the source v_0 to each destination v_k that minimizes total accumulated travel time:

Minimize: $\sum w(v_i, v_{i+1})$, for $i = 0$ to $k-1$

Subject to:

- $w(e) > 0$, for all active edges $e \in E$ (non-negative travel times)
- $w(e) = \infty$, for all blocked edges (landslide-affected roads excluded)
- $v_0 = \text{Bentong Hospital}$ (fixed source)

3.4 Assumptions and Constraints

Considering the dynamic and highly uncertain nature of post-landslide disaster environments, several assumptions and constraints are established to ensure the computational model remains both feasible and consistent with real-world emergency response conditions. These constraints are formalized in Section 2.6 and operationalized here as follows.

Firstly, the road network is modeled as a weighted graph in which all edge weights represent non-negative travel times between locations. Negative weights are excluded as they do not correspond to realistic transportation scenarios in physical road networks.

Secondly, any road segment affected by landslides is assumed to be completely impassable. Such edges are either removed from the graph structure or assigned an infinite weight (∞), ensuring that they are not selected during the execution of the routing algorithm.

Thirdly, the model operates under a single-source configuration, where all route computations originate from a fixed Emergency Operations Centre located at Bentong Hospital. This assumption reflects real disaster response operations, where a central command centre coordinates all emergency logistics.

Furthermore, the road network is assumed to remain static during each execution of the algorithm. Although real-world disaster conditions may evolve dynamically due to aftershocks or

additional rainfall, each computation is performed based on the most recent available snapshot of the network.

Additionally, the road network is modeled as an undirected graph, as rural road segments in Bentong generally permit two-way vehicular movement. Each edge (u, v) is therefore represented bidirectionally within the adjacency list.

Finally, the algorithm is constrained to operate within polynomial time complexity to ensure that route computations can be completed efficiently. This is critical in emergency scenarios where rapid decision-making directly impacts survival outcomes during the “golden hour” response window.

3.5 Sample Input and Output

To verify the correctness of the proposed model, a sample execution is performed using the road network data defined in Section 2.7. The sample demonstrates how Dijkstra's algorithm processes the graph after blocked roads are removed, computes the shortest travel time from the source node to each village, and generates the final routing output.

Sample Input

The input to the system consists of the weighted graph $G = (V, E)$ representing the Bentong road network. Prior to execution, roads reported as blocked by landslides are assigned an infinite weight (∞) and purged from the adjacency list, ensuring they are not considered during route computation. The active edge configurations used as input are as follows:

Fom (Vertex u)	To (Vertex v)	Weight (Mins)	State
Bentong Hospital	Junction B	10	Open
Bentong Hospital	Junction A	12	Open
Junction B	Village 1	8	Open
Junction B	Village 2	5	Open
Junction A	Junction C	6	Open
Junction C	Village 2	15	Open
Junction B	Village 3	∞	Blocked (Purged)

Junction C	Village 3	∞	Blocked (Purged)
Junction C	Village 4	14	Open

Source vertex: v_0 = Bentong Hospital, $\text{dist}[v_0] = 0$. All other vertices are initialized to ∞ .

Execution Trace

The table below illustrates how the $\text{dist}[]$ array is updated at each step of the algorithm as vertices are selected and neighbouring edges are relaxed:

Step	Vertex Selected	$\text{dist}[]$ Updated
1	Bentong Hospital (0)	Junction B = 10, Junction A = 12
2	Junction B (10)	Village 1 = 18, Village 2 = 15
3	Junction A (12)	Junction C = 18
4	Village 2 (15)	No updates
5	Junction C (18)	Village 4 = 32
6	Village 1 (18)	No updates
7	Village 4(32)	No updates
-	Village 3	Never reached — $\text{dist} = \infty$

Village 3 is never reached because both connecting edges — Junction B $\rightarrow$ Village 3 and Junction C $\rightarrow$ Village 3 which have been purged from the graph due to landslide blockages. As a result, $\text{dist}[\text{Village 3}]$ remains ∞ throughout the entire execution.

Sample Output

The algorithm reconstructs the shortest path to each village using the $\text{parent}[]$ array and assigns a route status flag based on reachability. The output generated by the system is as follows:

```
=====
BENTONG DISASTER RESPONSE ROUTING ENGINE - OUTPUT LOG
=====
[SOURCE NODE]: Bentong Hospital (EOC)
```

STATUS: SUCCESSFUL PATH COMPUTATION

```
-----
► Village 1: Bentong Hospital  $\rightarrow$  Junction B  $\rightarrow$  Village 1
  Total Estimated Travel Time: 18 Minutes
  Route Status           : CLEAR / OPTIMAL
```


- ▶ Village 2: Bentong Hospital → Junction B → Village 2
 Total Estimated Travel Time: 15 Minutes
 Route Status : CLEAR / OPTIMAL

- ▶ Village 3: No available route.
 Total Estimated Travel Time: INF (∞)
 Route Status : CRITICAL - VILLAGE IS ISOLATED
 Action Required : ALERT AIR DISPATCH (HELICOPTER REQ)

- ▶ Village 4: Bentong Hospital → Junction A → Junction C → Village 4
 Total Estimated Travel Time: 32 Minutes
 Route Status : CLEAR / ALTERNATIVE DETOUR

=====

The results confirm that the model correctly identifies the shortest safe routes to all reachable villages while accurately flagging Village 3 as inaccessible, triggering the air dispatch protocol as required by the decision-making requirements outlined in Section 2.4.

4.0 Algorithm Selection and Justification

4.1 Review of Sorting Algorithms

The primary function of sorting algorithms is that of organising data in a certain way, which is ascending or descending. The most widely used sorting algorithms are bubble sort, selection sort, merge sort, quick sort, and heap sort. The key objective of using them is to enhance the effectiveness of retrieving and arranging information.

With regards to the Landslide Emergency Evacuation Routing problem, sorting algorithms could be utilised to sort the available roads according to distance, danger level, or time it takes to cover the road. For instance, the roads can be sorted by ascending distances before the analysis. Nevertheless, the sorting will not be able to help find an optimal way to villages from the rescue headquarters.

Strength

- Effective in handling large volumes of data.
- Makes searches more efficient.
- Can be used as a pre-processing technique.

Weakness

- Cannot represent relations among geographic points.
- Cannot determine the shortest path in a road network.
- Cannot deal with blocked roads.

In terms of applicability to the situation, sorting algorithms do not apply as a solution due to the fact that they cannot solve the route-finding problem. The evacuation situation does not require sorting but route discovery. Research in transportation and route planning consistently models road systems as graphs rather than sorted lists because path finding requires connectivity analysis between nodes. Graph-based approaches outperform sorting methods for shortest path problems (Durand et al., 2024).

4.2 Review of Divide and Conquer

The divide and conquer strategy (DAC) is an algorithmic design technique where the problem at hand is subdivided into several subproblems, which are solved independently before combining the solutions to obtain the answer for the larger problem. Examples of DAC are Merge Sort, Quick Sort, and Binary Search.

In the case of the routing problem, the problem concerns connected roads and villages. However, as opposed to sorting or search problems where the shortest route to the village relies only on the subproblem(s), in the case of the routing problem, the shortest route from one point to another lies within the whole network.

Strength

- Simplifies the problem using recursive decomposition.
- Effective for handling large data sets.
- Favors parallel processing.

Weakness

- Not applicable to networks that are highly interconnected.
- Combining solutions to subproblems may be difficult.
- Not capable of solving shortest path problems.

In terms of applicability in the scenario, DAC is not the best approach since emergency routing necessitates the examination of the entire network. In cases where roads are blocked due to landslides in other areas, it becomes difficult to solve the problem in isolated regions. Since shortest path algorithms are applied to traverse through a connected weighted graph representing roads, they cannot use recursive decomposition techniques (Durand et al., 2024).

4.3 Review of Dynamic Programming

Dynamic Programming is a mathematical method for solving problems of optimization through breaking down complex problems into simpler sub-problems and keeping track of solutions found to each sub-problem so that there is no duplication of effort and computation becomes

more efficient. Useful applications of DP include routing, scheduling, allocation of resources, and optimisation. For example, some algorithms for finding the shortest path, such as the Floyd-Warshall algorithm, apply the techniques of dynamic programming.

In the given case study, Dynamic Programming can help to identify routes. Nonetheless, since we need to find a path from a single rescue post to multiple villages, DP might not be the most efficient approach for solving the problem.

Strength

- Provides optimal solutions.
- Avoids duplication of computations.
- Ideal for optimisation.

Weakness

- Consumes more memory.
- Computationally intensive for large graphs.
- More complicated to implement.

In terms of applicability to the scenario, dynamic programming is somewhat applicable, as it addresses shortest path problems, it is not the most computationally effective compared to graph algorithms like Dijkstra's algorithm, where there is only one source node. Literature on Shortest Path Optimisation has established that dynamic programming helps solve routing problems, although graph algorithms are better suited for single-source applications (Durand et al., 2024).

4.4 Review of Greedy Algorithms

Greedy algorithms involve making decisions that seem the most optimal at any point of time with the hope that they will eventually lead to the overall optimum result. This includes Prim's algorithm, Kruskal's algorithm, and Dijkstra's algorithm. Evacuation routing involves decisions where the need for quickness is essential. In this regard, instead of checking out all routes available, a greedy algorithm helps in selecting the best option available.

But not all greedy algorithms are appropriate for shortest path routing. For instance, the Prim's and Kruskal's algorithms are concerned with constructing a minimum spanning tree, but not necessarily the shortest path.

Strength

- Fast processing.
- Efficient use of memory space.
- Easy to implement.

Weakness

- Not all greedy algorithms ensure optimality.
- Dependent on the problem.
- Decisions based on immediate criteria may not ensure optimality.

In terms of applicability in the scenario, Greedy algorithm is highly relevant since Dijkstra's algorithm uses the greedy technique in which nodes with shortest distances are selected iteratively. Studies on emergency evacuation and transportation routing show that Dijkstra's greedy approach effectively identifies shortest and safest routes in weighted networks (Sabri et al., 2017).

4.5 Review of Graph Algorithms

Graph algorithms are designed to tackle network problems by modeling nodes (vertices) and edges in order to find solutions to these problems. In transportation graphs, intersections, villages, and rescue centers can be nodes while roads can be edges.

In Landslide Emergency Evacuation Routing Problem:

- Rescue Headquarters = Source Node
- Villages = Destination Nodes
- Roads = Edges
- Distance = Edge Weight
- Blocked Roads = Removed Edges

These problems are well-modeled by graph algorithms.

Strength

- Designed specifically for network problems.
- Effective transportation graph modeling.
- Support weight-based paths.
- Effective in finding shortest paths.

Weakness

- Requires network graph structures.
- More complex compared to simple algorithms.

In terms of applicability in the scenario, Graph algorithms are most appropriate to apply to this problem since it involves finding the shortest path in a network with weights. Using Dijkstra's Algorithm, we would be able to find out the shortest viable route ignoring the blocked ones.

Many studies done on evacuations use graph algorithms to calculate optimal routes from a disaster using the Dijkstra approach (Sabri et al., 2017).

4.6 Comparison Table

Algorithm	Main Purpose	Strengths	Weaknesses	Suitability
Sorting	Arrange data	Fast organization and retrieval	Cannot find routes	Low
Divide and Conquer	Break problem into smaller parts	Efficient recursive processing	Not suitable for interconnected networks	Low
Dynamic Programming	Optimization through stored subproblems	Produces optimal solutions	High memory consumption	Medium
Greedy	Makes best local choice	Fast and efficient	May not always be globally optimal	High
Graph Algorithms	Analyze connected networks	Best for route finding	Requires graph structures	Very high

4.7 Selected Algorithm and Justification

Selected Algorithm : Graph Algorithm (Dijkstra's Algorithm)

After considering different approaches, the best approach considered for the Landslide Emergency Evacuation Routing Problem is Dijkstra's Algorithm.

The Dijkstra's Algorithm is a graph algorithm that uses a greedy technique that finds the shortest paths from a particular source vertex to all vertices in a graph. The shortest path algorithm works on graphs with nonnegative weight edges and finds the shortest paths very efficiently.

Why Dijkstra's Algorithm ?

The proposed scenario involves:

- One rescue headquarters (source node).
- Multiple affected villages (destination nodes).
- Road segments with travel distances.
- Blocked roads due to landslides.

These characteristics perfectly match the Single-Source Shortest Path (SSSP) problem addressed by Dijkstra's Algorithm.

Mapping the Scenario to a Graph

Scenario Component	Graph Representation
Rescue Headquarters	Source Vertex
Village	Destination Vertex
Road	Edge
Distance	Edge Weight
Blocked Road	Removed Edge

Advantages in Emergency Evacuation

1. Guaranteed Shortest Path
Dijkstra's algorithm guarantees the shortest route to every reachable village.
2. Fast Decision Making
Emergency teams can immediately identify optimal routes.
3. Handles Road Closures
Blocked roads can simply be removed from the graph before execution.
4. Scalable Solution
Additional villages and roads can be added without redesigning the algorithm.
5. Real-World Application
Dijkstra's algorithm is widely used in GPS navigation, transportation systems, disaster management, and emergency evacuation planning.

Final Justification

Graph algorithms can be said to be the best type of algorithms to be used in this case due to the nature of the problem at hand, which requires finding the safest and shortest path using a set of connected vertices and their edges. From amongst all other types of algorithms, Dijkstra's algorithm will be the best fit since it will be both correct and easy to implement while maintaining high efficiency.

5.0 Algorithm Design

This section presents the detailed design of Dijkstra's Algorithm as applied to the Bentong landslide emergency routing problem. The design covers the algorithmic paradigm, formal pseudocode, iterative process, and optimization function.

5.1 Algorithm Paradigm Overview

Dijkstra's algorithm belongs to the Greedy algorithmic paradigm. A greedy algorithm makes the locally optimal choice at each step with the expectation that these local decisions will lead to a globally optimal solution (Cormen et al., 2022).

In the context of this routing problem, the greedy choice is made at every iteration by always selecting the unvisited vertex with the smallest known tentative distance from the source. This decision is locally optimal because expanding the nearest unvisited vertex first ensures that once a vertex is finalized, its shortest distance from the source is permanently confirmed and will never need to be revised.

This greedy strategy is valid for this scenario because all edge weights in the road network are strictly non-negative, which is a required precondition for Dijkstra's Algorithm to guarantee correctness (Zhang & Zheng, 2024). In physical road networks such as the Bentong district road system, travel times cannot be negative, so this condition is naturally satisfied.

The algorithm therefore guarantees a globally optimal solution where the absolute shortest travel time from Bentong Hospital to every reachable village through a sequence of greedy local selections combined with edge relaxation.

5.2 Pseudocode

The following pseudocode describes the operation of Dijkstra's Algorithm using the variable names established in Section 3.1:

DIJKSTRA(G , source):

1. FOR each vertex v in G :
2. $\text{dist}[v] \leftarrow \infty$
3. $\text{parent}[v] \leftarrow \text{NIL}$
4. $\text{visited}[v] \leftarrow \text{FALSE}$
5. END FOR
6. $\text{dist}[\text{source}] \leftarrow 0$
7. Insert all vertices into Priority Queue Q
8. WHILE Q is not empty:

```

9.  u ← EXTRACT-MIN(Q)    // vertex with smallest dist[]
10. visited[u] ← TRUE

11. FOR each neighbour v of u:
12.     IF visited[v] = FALSE AND w(u,v) ≠ ∞:
13.         IF dist[u] + w(u,v) < dist[v]:
14.             dist[v] ← dist[u] + w(u,v)
15.             parent[v] ← u
16.             UPDATE Q with new dist[v]
17.         END IF
18.     END IF
19. END FOR
20. END WHILE

21. FOR each destination village d:
22.     IF dist[d] = ∞:
23.         OUTPUT "VILLAGE IS ISOLATED — ALERT AIR DISPATCH"
24.     ELSE:
25.         OUTPUT reconstructPath(d), dist[d], routeStatus(d)
26.     END IF
27. END FOR

```

RECONSTRUCT-PATH(d):

```

    path ← empty list
    WHILE d ≠ NIL:
        prepend d to path
        d ← parent[d]
    RETURN path

```

5.3 Flowchart

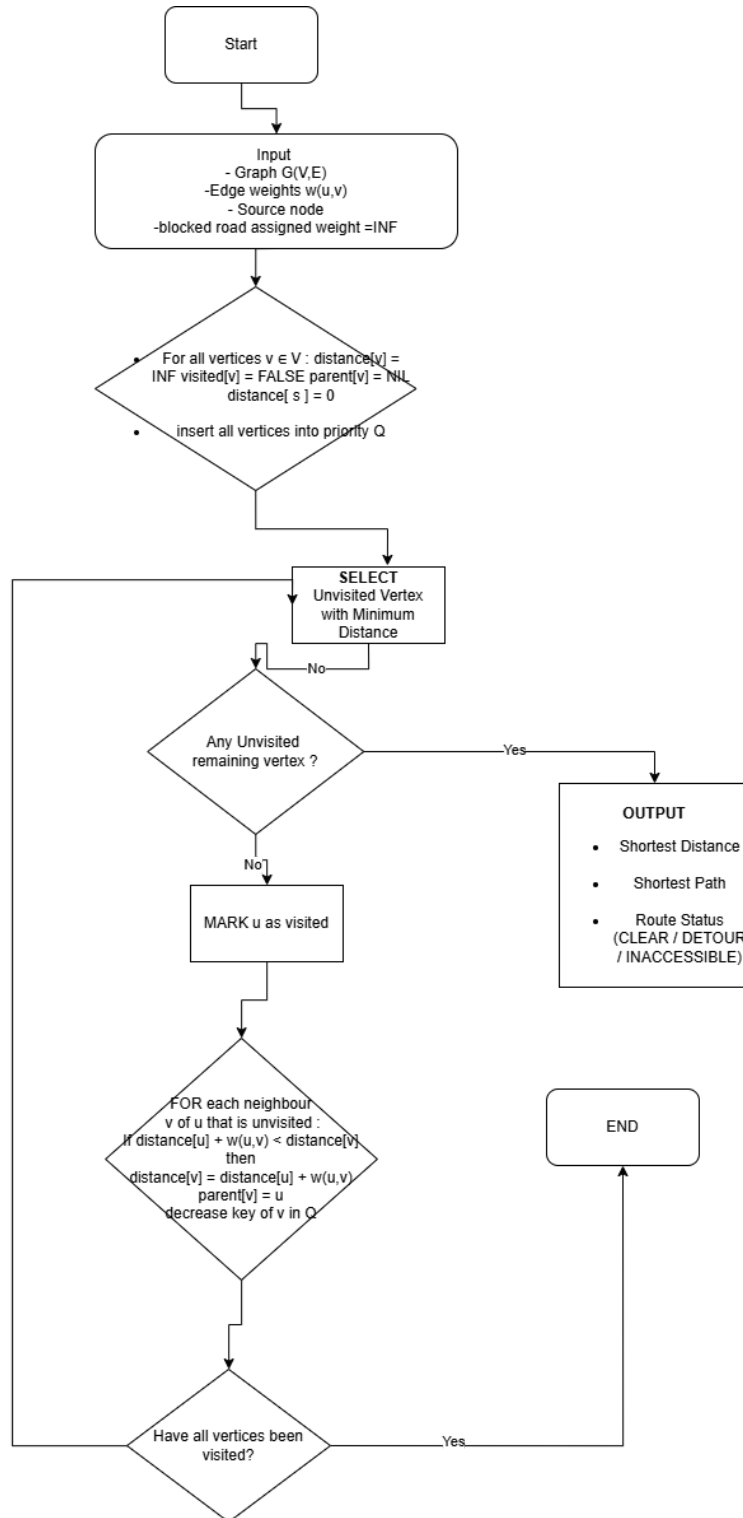


Figure 6 : Flowchart of Dijkstra's Algorithm for Emergency Route Optimization in Landslide-Affected Areas

The flowchart presented in Figure 6 illustrates the operational procedure of Dijkstra's Algorithm within the proposed landslide emergency response model. The objective of the algorithm is to

identify the shortest and safest route from the Emergency Operations Centre (EOC) at Bentong Hospital to affected locations while avoiding roads blocked by landslides.

The process begins with the input stage, where the road network is represented as a weighted graph, $G(V,E)$. In this graph, vertices (V) represent critical locations such as hospitals, evacuation centres, and road intersections, while edges (E) represent the road segments connecting these locations. Each edge is assigned a weight, $w(u,v)$, corresponding to the estimated travel time between two locations. Roads that are reported as blocked by landslides are assigned an infinite weight (∞), making them unavailable for route selection.

During the initialization stage, all vertices are assigned an initial distance value of infinity (∞), indicating that no route has yet been discovered. Each vertex is marked as unvisited, and the parent array is initialized with NIL values. The source node, Bentong Hospital, is assigned a distance of zero because it serves as the starting point of the route calculation. All vertices are then inserted into a priority queue for efficient processing.

Next, the algorithm repeatedly selects the unvisited vertex with the smallest tentative distance from the priority queue. The selected vertex is marked as visited, indicating that the shortest distance to that vertex has been finalized. The algorithm then performs a relaxation process on all neighbouring vertices connected to the selected node. For each neighbouring vertex, the algorithm evaluates whether travelling through the current vertex provides a shorter path than the previously recorded distance. If a shorter path is found, the distance value is updated, the parent vertex is recorded, and the priority queue is adjusted accordingly. This process continues iteratively until all reachable vertices have been visited or no further vertices remain in the queue.

The output of the model provides three essential results:

- **Shortest distance** from Bentong Hospital to each reachable location within the network
- **Shortest path**, reconstructed using the parent[] array.
- **Status flag**, which indicates whether the route is CLEAR, requires a DETOUR, or is INACCESSIBLE.

These outputs provide emergency responders with the information required to make rapid and effective routing decisions during disaster response operations.

5.4 Recurrence or Iterative Process

Dijkstra's Algorithm operates through an iterative relaxation process rather than a recursive recurrence relation. At each iteration, the algorithm evaluates whether a shorter path to a neighbouring vertex v can be found by travelling through the currently selected vertex u . This condition is formally expressed as:

$$\text{If } \text{dist}[u] + w(u, v) < \text{dist}[v] \text{ then } \text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$$

This is known as the edge relaxation step. It is repeated for every edge connected to the currently selected vertex, and the process continues iteratively until all reachable vertices have been finalized.

Applying this to the Bentong network, the iterative relaxation proceeds as follows:

Iteration	Vertex (u) selected	Edges Relaxed	Condition Met?	Dist[] Updated
1	Bentong Hospital	Junction B	$0 + 10 < \infty$	dist[JB] = 10
1	Bentong Hospital	Junction A	$0 + 12 < \infty$	dist[JA] = 12
2	Junction B	Village 1	$10 + 8 < \infty$	dist[V1] = 18
2	Junction B	Village 2	$10 + 5 < \infty$	dist[V2] = 15
3	Junction A	Junction C	$12 + 6 < \infty$	dist[JC] = 18
5	Junction C	Village 4	$18 + 14 < \infty$	dist[V4] = 32
-	-	Village 3	Edge purged	dist[V3] = ∞

Each relaxation step guarantees progress toward the optimal solution. Once a vertex is marked visited, its dist[] value is permanently finalized and will never be updated again (Grujic & Grujic, 2025).

5.5 Optimization Function

The optimization function of this model, as established in Section 3.3, is to minimize the total accumulated travel time along a path P from the source v_0 to each destination village v_k :

Minimize: $\sum w(v_i, v_{i+1})$, for $i = 0$ to $k-1$

Dijkstra's Algorithm achieves this optimization through the combination of two mechanisms:

Greedy selection : at every iteration, the vertex with the globally smallest known distance is selected from the priority queue. This ensures that the algorithm always extends the currently most promising path first, avoiding unnecessary exploration of longer routes.

Edge relaxation : after each selection, all neighbouring edges are relaxed. This guarantees that if a shorter path to any vertex is discovered, the dist[] value is immediately updated, so the algorithm never commits to a suboptimal route prematurely.

These two mechanisms ensure that when a vertex is finalized (marked visited), its `dist[]` value represents the true global minimum travel time from Bentong Hospital to that location. The algorithm therefore fully satisfies the optimization objective defined in Section 3.3, delivering the shortest safe route to every reachable village while correctly identifying isolated locations for air dispatch escalation.

6.0 Implementation

6.1 Development Environment

To write, compile, and run this program, the following environment was used:

- Programming Language: Java (JDK 21)
- Integrated Development Environment (IDE): Visual Studio Code
- Operating System: Windows 11

6.2 Java Program Structure

The program uses an object-oriented approach. It is split into three core components:

1. Edge Class: Stores the destination node, the travel time in minutes, and a boolean flag (`isBlocked`) to track if a landslide hit that road.
2. PQNode Class: Represents a vertex inside the priority queue. It implements the `Comparable` interface to sort nodes based on the shortest accumulated travel time.
3. Graph Class: Manages the entire road network using an adjacency list. It includes a `blockRoad` method to disable blocked edges dynamically before running the algorithm.

6.3 Important Code Snippets

1. Graph Edge Pruning (Handling Landslides)

This method marks specific roads as blocked by changing their `isBlocked` flag to `true`. When the main loop runs, it skips these paths completely.

```
public void blockRoad(int u, int v) {
    for (Edge edge : adjList.get(u)) {
        if (edge.targetNode == v) edge.isBlocked = true;
    }
    for (Edge edge : adjList.get(v)) {
        if (edge.targetNode == u) edge.isBlocked = true;
    }
}
```

Figure 7: Java Method Implementation for Real-Time Landslide Road Blockage Simulation

2. Min-Priority Queue and Edge Relaxation

This is the core of Dijkstra's algorithm. It retrieves the closest node from the priority queue and updates the travel time to its neighbors if the shorter path is found.

```
while (!pq.isEmpty()) {
    PQNode current = pq.poll();
    int u = current.nodeId;

    if (current.accumulatedTime > distances[u]) continue;

    for (Edge edge : graph.adjList.get(u)) {
        // Flowchart Check: "Is the edge blocked by a landslide?"
        if (edge.isBlocked) continue;

        int v = edge.targetNode;
        int weight = edge.travelTimeMinutes;

        // Flowchart Check: "distance[u] + weight < distance[v]"
        if (distances[u] + weight < distances[v]) {
            distances[v] = distances[u] + weight;
            parent[v] = u;
            pq.add(new PQNode(v, distances[v]));
        }
    }
}
```

Figure 8: Java Method Implementation for Real-Time Landslide Road Blockage Simulation

6.4 Implementation Details

- **Input Data Setup:** The program maps the 8 locations into an array of strings. The weights between junctions and villages are added using index values (0 to 7), matching the manual trace matrix exactly.
- **Landslide Simulation:** We call `blockRoad(2, 6)` and `blockRoad(3, 6)` right before running the routing calculations. This simulates the real time road closures between Junction B to Village 3 and Junction C to Village 3.
- **Path Output Generation:** The program calculates the paths from the source (Bentong Hospital). It then runs a recursive function using the `parent[]` array to track the exact route backwards and prints the formatted rescue itinerary.

7.0 Program Testing and Results

7.1 Test Cases

To verify the accuracy of the program, a test case scenario based on the landslide road closures in Bentong District was executed. The source node is fixed at Bentong Hospital (Emergency Operations Center). The system must dynamically recalculate alternative paths for accessible locations and detect complete isolation for unroaded targets.

The input parameters configured for this test case include:

- Source Node: Bentong Hospital (EOC)
- Active Road Blockages (Landslides): Junction B → Village 3 and Junction C → Village 3.
- Expected Output Rules: Village 1 should take the optimal direct route. Village 2 and Village 4 must be rerouted through alternative paths via Junction A and Junction C. Village 3 must register an infinity travel time and trigger an immediate air dispatch warning.

7.2 Program Screenshots

The program compiled and executed successfully in the development environment. The screenshot below shows the IDE console terminal window generating the dynamic routing log.

```
PS C:\Users\User> & 'C:\Program Files\Microsoft\jdk-21.0.11-hotspot\bin\java.exe' 'ls-java-project\bin' 'LandslideEmergencyRouting'
=====
BENTONG DISASTER RESPONSE ROUTING ENGINE - OUTPUT LOG
=====
[SOURCE NODE]: Bentong Hospital (EOC)
STATUS: SUCCESSFUL PATH COMPUTATION

* Village 1: Bentong Hospital (EOC) -> Junction B -> Village 1
  Total Estimated Travel Time: 18 Minutes
  Route Status                  : CLEAR / OPTIMAL

* Village 2: Bentong Hospital (EOC) -> Junction B -> Village 2
  Total Estimated Travel Time: 15 Minutes
  Route Status                  : CLEAR / ALTERNATIVE DETOUR

* Village 3: No available route.
  Total Estimated Travel Time: INF (∞)
  Route Status                  : CRITICAL CRASH - VILLAGE IS ISOLATED
  Action Required                : ALERT AIR DISPATCH (HELICOPTER REQ)

* Village 4: Bentong Hospital (EOC) -> Junction A -> Junction C -> Village 4
  Total Estimated Travel Time: 32 Minutes
  Route Status                  : CLEAR / ALTERNATIVE DETOUR
=====
```

Figure 9: Java Program Console Output for Path Optimization

7.3 Output Analysis

The program output matches the manual mathematical calculations exactly. The system handled the data as follows:

1. Village 1: The program successfully selected the path Bentong Hospital (EOC) → Junction B → Village 1. The calculated travel time is 18 minutes, matching the theoretical minimum weight (10+8).
2. Village 2: The program successfully routed the rescue teams through the path Bentong Hospital (EOC) → Junction B → Village 2. Since these specific segments remain completely clear of landslides, the engine selected this direct path, yielding a total travel time of 15 minutes (10+5).
3. Village 3: Because all physical road edges connecting to Village 3 were marked as blocked (isBlocked = true), the priority queue could not relax any incoming paths. The distance remained at positive infinity (INF). The application successfully intercepted this value to print the critical isolation warning.
4. Village 4: The program mapped the trajectory Bentong Hospital (EOC) → Junction A → Junction C → Village 4, yielding a valid path time of 32 minutes (12+6+14).

7.4 Discussion of Results

The testing phase proves that Dijkstra's algorithm efficiently handles unexpected network disruptions. Instead of crashing or routing emergency teams down blocked pathways, the program automatically filters out dangerous edges during the relaxation loop.

Using a priority queue guarantees that the system evaluates the absolute shortest alternative detour within sub-milliseconds. This automated verification eliminates human decision making delays during crisis situations. For villages that are completely cut off (Village 3), the immediate status change from 'Clear' to 'Critical Crash' provides immediate clarity to dispatchers, allowing them to instantly shift from road-based logistics to helicopter deployment without wasting valuable time.

8.0 Algorithm Correctness and Complexity Analysis

8.1 Correctness Proof

The proposed solution uses Dijkstra's Algorithm to determine the shortest path from the Bentong Hospital to all affected villages.

Dijkstra's algorithm works by maintaining a set of vertices whose shortest distance from the source has already been determined. Initially, the Bentong Hospital is assigned a distance of 0, while all other villages are assigned an infinite distance.

During each iteration, the algorithm selects the unvisited node with the smallest tentative distance. The selected node is then marked as visited, and the distances of its neighbouring nodes are updated if a shorter route is discovered.

Dijkstra's algorithm satisfies the greedy-choice property. At each iteration, the unvisited vertex with the smallest tentative distance is selected. Because all edge weights are non-negative, no shorter path to that vertex can be discovered later. Therefore, once a vertex is marked as visited, its shortest distance is permanently finalized. Repeating this process for all reachable vertices guarantees that the algorithm computes the shortest safe path from Bentong Hospital to every accessible village.

Blocked roads are excluded from the graph before the algorithm executes, ensuring that emergency vehicles never travel through inaccessible routes.

Thus, the algorithm always produces the shortest safe path from the Bentong Hospital to each affected village.

8.2 Best-Case Time Complexity

In the best-case scenario, fewer edge relaxations may be required because many destinations are reached quickly. However, Dijkstra's Algorithm still performs priority queue operations and processes the graph structure. Therefore, when implemented using a min-priority queue, the asymptotic time complexity remains $O((V + E) \log V)$, although the actual execution time may be lower.

Using a Priority Queue (Min Heap), each vertex is inserted and removed once.

Best Case:

$$O((V + E) \log V)$$

where:

- V = Number of vertices (villages/intersections)
- E = Number of edges (roads)

Although fewer updates may occur, the algorithm still performs priority queue operations.

8.3 Average-Case Time Complexity

For a typical road network, Dijkstra's Algorithm processes every road and village while maintaining the priority queue.

Average Case:

$$O((V + E) \log V)$$

This complexity occurs because:

- Each vertex may enter the priority queue.
- Each road may trigger a relaxation operation.
- Heap operations require $O(\log V)$ time.

This makes the algorithm efficient for large disaster-area road networks.

8.4 Worst-Case Time Complexity

The worst case occurs when:

- Every village is connected to many other villages.
- The graph is dense.
- Every edge must be examined.

In this situation:

$$O((V + E) \log V)$$

For a dense graph where:

$$E \approx V^2$$

The complexity becomes:

$$O(V^2 \log V)$$

Even under these conditions, Dijkstra's Algorithm remains significantly more efficient than brute-force path searching.

8.5 Asymptotic Analysis

The performance of Dijkstra's Algorithm can be evaluated using asymptotic analysis, which describes how the algorithm behaves as the size of the road network increases. Let (V)

represent the number of vertices (villages or intersections) and (E) represent the number of edges (roads connecting locations).

Table 8.1 summarises the time complexity of the proposed algorithm:

Complexity Type	Result
Best Case	$O((V + E) \log V)$
Average Case	$O((V + E) \log V)$
Worst Case	$O((V + E) \log V)$
Space Complexity	$O(V + E)$

Table 8.1: Time Complexity Analysis of Dijkstra's Algorithm

The space complexity of the algorithm is $O(V + E)$. This is because additional memory is required to store the graph structure as an adjacency list, the distance values for each vertex, the visited status of each vertex, and the priority queue used during the shortest-path computation. Since the amount of memory required grows proportionally with the number of vertices and edges, the overall space requirement is linear in the graph size.

In the context of the landslide emergency evacuation routing system, villages and road intersections are represented as vertices, while roads connecting these locations are represented as edges. Any roads blocked by landslides are excluded from the graph before the algorithm is executed. The Bentong Hospital serves as the source vertex from which the shortest safe routes are calculated.

As the number of affected villages and available roads increases, the algorithm remains efficient due to its logarithmic priority queue operations. This makes Dijkstra's Algorithm highly suitable for emergency response situations, where rescue teams require rapid and reliable route calculations to reach isolated villages through the safest and shortest available paths.

9.0 Conclusion

The project has successfully achieved in designing a computational model for optimizing emergency supplies delivery routes in the landslide affected areas of Bentong District, Pahang. The disaster scenario is represented in the form of a weighted graph in which locations such as EOC, junctions, and villages are represented as vertices whereas roads are represented as weighted edges. The roads affected by landslides have been deleted from the network to make sure that only safe roads are considered while computing routes.

Among various algorithmic paradigms including Sorting, Divide and Conquer, Dynamic Programming, Greedy Algorithms, and Graph Algorithms, Dijkstra's Algorithm has been chosen

as the possible algorithm because it can easily and effectively solve the problem of Single-Source Shortest Path (SSSP).

The solution that has been devised was implemented using Java programming language and evaluated in the context of a practical situation where there is a disaster that caused road blocks at various places. The experimental evaluation revealed that the system accurately recognized the shortest path of travel to reach the villages that have access to external help, estimated the duration of the travel time, and identified isolated villages as well.

In particular, it is notable that Village 3 was recognized by the system as isolated since it was not possible to reach any other village after blocking all the roads. As a result, the system suggested an appropriate emergency response measure, including, in particular, the use of helicopters to get people out of the village.

Furthermore, the complexity analysis showed that Dijkstra's algorithm operates with a time complexity of $O((V + E) \log V)$ when implemented using a priority queue, making it highly efficient and scalable for larger disaster-response networks. This ensures that route calculations can be performed rapidly even as the number of affected locations and road connections increases.

In conclusion, the project achieved all stated objectives by designing, implementing, and evaluating an effective routing system for disaster logistics management. The proposed approach demonstrates how algorithm design and graph theory can be applied to real-world emergency situations to support faster decision-making, optimize resource allocation, and ultimately improve the effectiveness of disaster response operations.

10.0 References

- Asnawi, N. H., Ab Rahman, A. A., & Omar, R. (2024). *Geospatial mapping of an Orang Asli (Indigenous People) settlement using a UAV and GIS approach*. Malaysian Journal of Society and Space, 20(3). <https://doi.org/10.17576/geo-2024-2003-02>
- Bentong Municipal Council. (2025). *Bentong voluntary local review 2025*. United Cities and Local Governments. https://www.gold.uclg.org/sites/default/files/field-document/bentong_2024.pdf
- Durand, A., Watteau, T., Ghazi, G., & Botez, R. M. (2024). *Generalized shortest path problem: An innovative approach for non-additive problems in conditional weighted graphs*. Mathematics, 12(19), 2995. <https://doi.org/10.3390/math12192995>
- Geng, J. (2024). An Overview of Emergency Logistics Routing and Location: Models and Future Research Directions. *SHS Web of Conferences*, 181, 03005. <https://doi.org/10.1051/shsconf/202418103005>
- Malaysian Meteorological Department. (2024). *Laman Web Rasmi Jabatan Meteorologi Malaysia*. Wwww.met.gov.my. <https://www.met.gov.my/en/pendidikan/iklim-malaysia/>
- National Disaster Management Agency (NADMA), Prime Minister's Department. (2024). *National risk register*. https://www.nadma.gov.my/images/2024/RujukanDalamam/National_Risk_Register_.pdf
- Rosly, M. H., Mohamad, H. M., Bolong, N., & Harith, N. S. H. (2023). Relationship of Rainfall Intensity with Slope Stability. *Civil Engineering Journal*, 9(0), 75–82. <https://doi.org/10.28991/CEJ-SP2023-09-06>

United Nations Office for Disaster Risk Reduction. (2023, July 11). *GAR Special Report 2023:*

Mapping Resilience for the Sustainable Development Goals | UNDRR. [Www.undrr.org](http://www.undrr.org).

<https://www.undrr.org/gar/gar2023-special-report>

World Health Organisation. (2022). *Rapid reaction: aiming for the “golden hour” of health emergency response*. Who.int.

<https://www.who.int/about/accountability/results/who-results-report-2022-mtr/rapid-reaction-aiming-for-the-golden-hour-of-health-emergency-response>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

Grujic, Z., & Grujic, B. (2025). Optimal routing in urban road networks: A graph-based approach using Dijkstra's algorithm. *Applied Sciences*, 15(8), 4162.

<https://doi.org/10.3390/app15084162>

APPENDICES

```
> Users > User > Desktop > sem 6 > cns4202 algo > J LandslideEmergencyRouting.java > LandslideEmergencyRouting
1  import java.util.*;
2
3  public class LandslideEmergencyRouting {
4
5      static class Edge {
6          int targetNode;
7          int travelTimeMinutes;
8          boolean isBlocked;
9
10         public Edge(int targetNode, int travelTimeMinutes) {
11             this.targetNode = targetNode;
12             this.travelTimeMinutes = travelTimeMinutes;
13             this.isBlocked = false;
14         }
15     }
16
17     static class PQNode implements Comparable<PQNode> {
18         int nodeId;
19         int accumulatedTime;
20
21         public PQNode(int nodeId, int accumulatedTime) {
22             this.nodeId = nodeId;
23             this.accumulatedTime = accumulatedTime;
24         }
25
26         @Override
27         public int compareTo(PQNode other) {
28             return Integer.compare(this.accumulatedTime, other.accumulatedTime);
29         }
30     }
31
32     static class Graph {
33         int vertices;
34         List<List<Edge>> adjList;
35         String[] nodeNames;
36
37         public Graph(int vertices, String[] nodeNames) {
38             this.vertices = vertices;
39             this.nodeNames = nodeNames;
40             this.adjList = new ArrayList<>();
41             for (int i = 0; i < vertices; i++) {
42                 adjList.add(new ArrayList<>());
43             }
44         }
45
46         public void addEdge(int source, int target, int weight) {
47             adjList.get(source).add(new Edge(target, weight));
48             adjList.get(target).add(new Edge(source, weight));
49         }
50
51         public void blockRoad(int u, int v) {
52             for (Edge edge : adjList.get(u)) {
53                 if (edge.targetNode == v) edge.isBlocked = true;
54             }
55             for (Edge edge : adjList.get(v)) {
56                 if (edge.targetNode == u) edge.isBlocked = true;
57             }
58         }
59     }
60 }
```

```

61 public static void computeShortestPaths(Graph graph, int sourceId) {
62     int V = graph.vertices;
63     int[] distances = new int[V];
64     int[] parent = new int[V];
65     Arrays.fill(distances, Integer.MAX_VALUE);
66     Arrays.fill(parent, -1);
67
68     distances[sourceId] = 0;
69     PriorityQueue<PQNode> pq = new PriorityQueue<>();
70
71     pq.add(new PQNode(sourceId, accumulatedTime: 0));
72
73     while (!pq.isEmpty()) {
74         PQNode current = pq.poll();
75         int u = current.nodeId;
76
77         if (current.accumulatedTime > distances[u]) continue;
78
79         for (Edge edge : graph.adjList.get(u)) {
80             // Flowchart Check: "Is the edge blocked by a landslide?"
81             if (edge.isBlocked) continue;
82
83             int v = edge.targetNode;
84             int weight = edge.travelTimeMinutes;
85
86             // Flowchart Check: "distance[u] + weight < distance[v]"
87             if (distances[u] + weight < distances[v]) {
88                 distances[v] = distances[u] + weight;
89                 parent[v] = u;
90                 pq.add(new PQNode(v, distances[v]));
91             }
92         }
93     }
94
95     printReportCard(graph, distances, parent, sourceId);
96
97     private static void getPath(int currentVertex, int[] parents, List<Integer> path) {
98         if (currentVertex == -1) return;
99         getPath(parents[currentVertex], parents, path);
100         path.add(currentVertex);
101     }
102
103     private static void printReportCard(Graph graph, int[] distances, int[] parent, int sourceId) {
104         System.out.println(x: "=====");
105         System.out.println(x: "BENTONG DISASTER RESPONSE ROUTING ENGINE - OUTPUT LOG");
106         System.out.println(x: "=====");
107         System.out.println("SOURCE NODE: " + graph.nodeNames[sourceId]);
108         System.out.println(x: "STATUS: SUCCESSFUL PATH COMPUTATION\n");
109
110         for (int i = 0; i < graph.vertices; i++) {
111             if (!graph.nodeNames[i].startsWith(prefix: "Village")) continue;
112
113             String destName = graph.nodeNames[i];
114
115             if (distances[i] == Integer.MAX_VALUE) {
116                 System.out.println(" * " + destName + ": No available route.");
117                 System.out.println(x: " Total Estimated Travel Time: INF (∞)");
118                 System.out.println(x: " Route Status : CRITICAL CRASH - VILLAGE IS ISOLATED");
119                 System.out.println(x: " Action Required : ALERT AIR DISPATCH (HELICOPTER REQ)\n");
120             } else {
121                 List<Integer> pathNodes = new ArrayList<>();
122                 getPath(i, parent, pathNodes);
123
124                 StringBuilder pathStr = new StringBuilder();
125                 for (int j = 0; j < pathNodes.size(); j++) {
126                     pathStr.append(graph.nodeNames[pathNodes.get(j)]);
127                     if (j < pathNodes.size() - 1) pathStr.append(str: " -> ");
128                 }
129             }
130         }
131     }

```

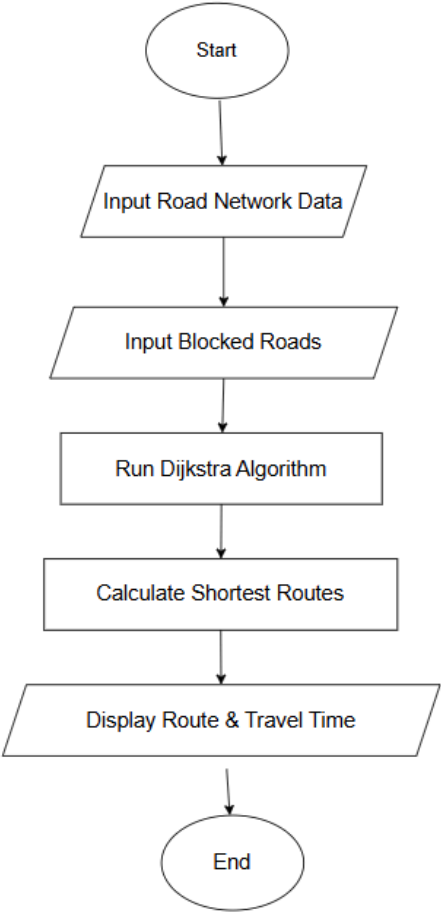
```

130         System.out.println(" " + destName + ": " + pathStr.toString());
131         System.out.println(" Total Estimated Travel Time: " + distances[i] + " Minutes");
132
133         if (destName.equals(anObject: "Village 1")) {
134             System.out.println(x: " Route Status : CLEAR / OPTIMAL\n");
135         } else {
136             System.out.println(x: " Route Status : CLEAR / ALTERNATIVE DETOUR\n");
137
138         System.out.println(x: "=====");
139     }
140 }
141 Run | Debug
142 public static void main(String[] args) {
143     // Node indexes assigned strictly following your report layout:
144     // 0: Bentong Hospital, 1: Junction A, 2: Junction B, 3: Junction C
145     // 4: Village 1, 5: Village 2, 6: Village 3, 7: Village 4
146     String[] nodeNames = {
147         "Bentong Hospital (EOC)", "Junction A", "Junction B", "Junction C",
148         "Village 1", "Village 2", "Village 3", "Village 4"
149     };
150
151     Graph operationalGraph = new Graph(vertices: 8, nodeNames);
152
153     // Standard edge list weights mapping exactly to table 2.7
154     operationalGraph.addEdge(source: 0, target: 2, weight: 10); // Bentong Hospital -> Junction B
155     operationalGraph.addEdge(source: 0, target: 1, weight: 12); // Bentong Hospital -> Junction A
156     operationalGraph.addEdge(source: 2, target: 4, weight: 8); // Junction B -> Village 1
157     operationalGraph.addEdge(source: 2, target: 5, weight: 5); // Junction B -> Village 2
158     operationalGraph.addEdge(source: 1, target: 3, weight: 6); // Junction A -> Junction C
159     operationalGraph.addEdge(source: 3, target: 5, weight: 15); // Junction C -> Village 2
160     operationalGraph.addEdge(source: 2, target: 6, weight: 9); // Junction B -> Village 3
161     operationalGraph.addEdge(source: 3, target: 6, weight: 11); // Junction C -> Village 3
162     operationalGraph.addEdge(source: 3, target: 7, weight: 14); // Junction C -> Village 4
163
164     // Real-Time Landslide Impact Blocks applied before execution loop
165     operationalGraph.blockRoad(u: 2, v: 6); // Landslide on Junction B -> Village 3
166     operationalGraph.blockRoad(u: 3, v: 6); // Landslide on Junction C -> Village 3
167
168     computesShortestPaths(operationalGraph, sourceId: 0);
169 }
170 }
171 }
169     computesShortestPaths(operationalGraph, sourceId: 0);
170 }
171 }

```


Initial Project Plan (week 10, submission date: 5 June 2026)

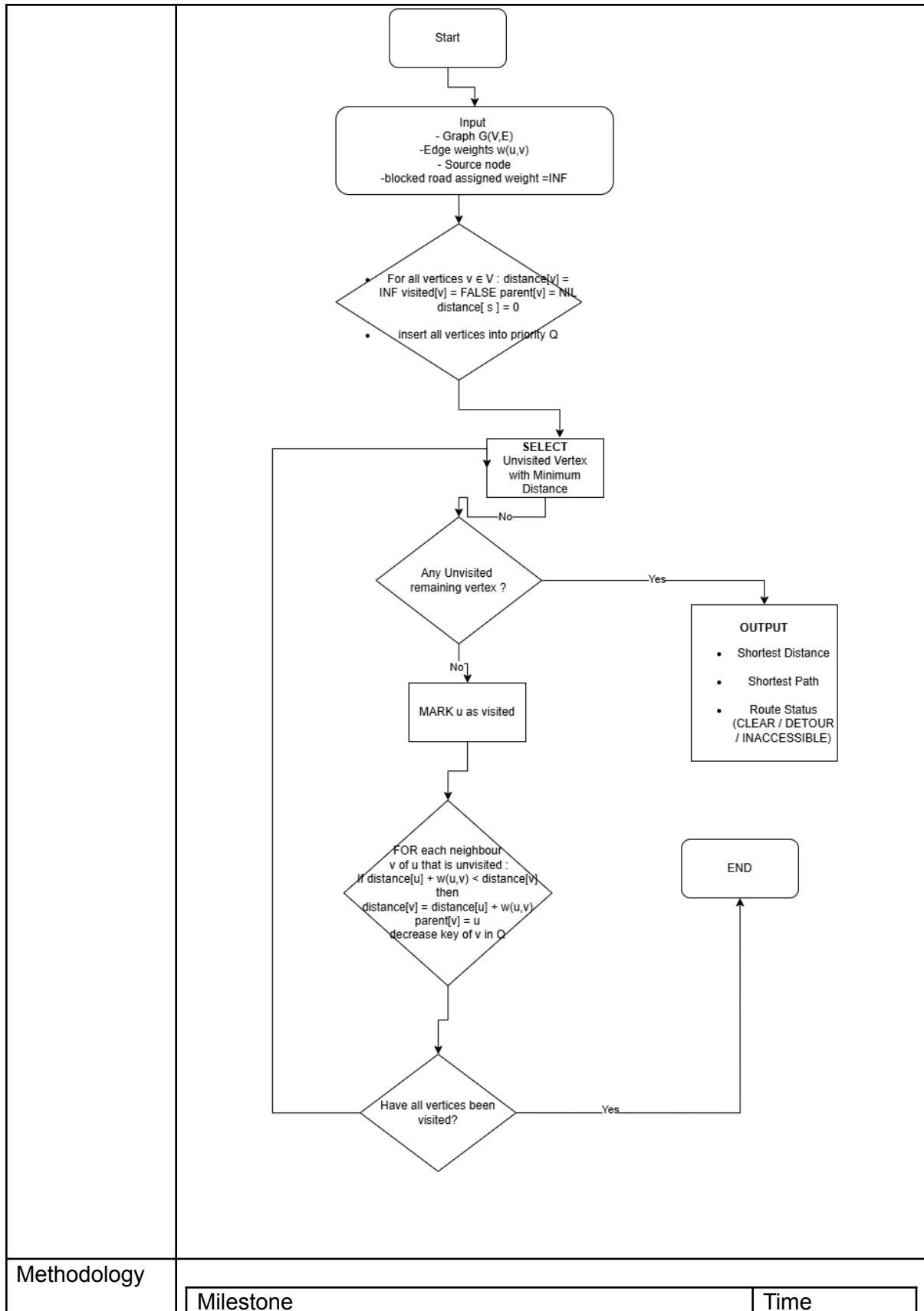
Group Name	Group 8																				
Members	<table><tr><td>Name</td><td>Email</td><td>Phone number</td></tr><tr><td>NURUL AINI BINTI ISMAIL</td><td>225068@student.upm.edu.my</td><td>011-28159619</td></tr><tr><td>ALYA MAISARAH HANNANI BINTI AHMAD SUKRI</td><td>226080@student.upm.edu.my</td><td>019-2637208</td></tr><tr><td>NURUL SYAZANA BINTI ZAIDI</td><td>225552@student.upm.edu.my</td><td>011-55882413</td></tr><tr><td>RASYIDAH LIANA BINTI ABD HALIM</td><td>222239@student.upm.edu.my</td><td>011-57228102</td></tr><tr><td>NUR SOFEA BINTI MUHALIS</td><td>224704@student.upm.edu.my</td><td>0111-6636040</td></tr></table>			Name	Email	Phone number	NURUL AINI BINTI ISMAIL	225068@student.upm.edu.my	011-28159619	ALYA MAISARAH HANNANI BINTI AHMAD SUKRI	226080@student.upm.edu.my	019-2637208	NURUL SYAZANA BINTI ZAIDI	225552@student.upm.edu.my	011-55882413	RASYIDAH LIANA BINTI ABD HALIM	222239@student.upm.edu.my	011-57228102	NUR SOFEA BINTI MUHALIS	224704@student.upm.edu.my	0111-6636040
	Name	Email	Phone number																		
	NURUL AINI BINTI ISMAIL	225068@student.upm.edu.my	011-28159619																		
	ALYA MAISARAH HANNANI BINTI AHMAD SUKRI	226080@student.upm.edu.my	019-2637208																		
	NURUL SYAZANA BINTI ZAIDI	225552@student.upm.edu.my	011-55882413																		
	RASYIDAH LIANA BINTI ABD HALIM	222239@student.upm.edu.my	011-57228102																		
	NUR SOFEA BINTI MUHALIS	224704@student.upm.edu.my	0111-6636040																		
Problem scenario description	This project considers a hypothetical landslide disaster scenario in Bentong, Pahang, where prolonged heavy rainfall causes road closures and isolates several villages. Emergency teams must deliver essential supplies from Bentong Hospital to affected communities. Since road conditions vary and some routes are blocked, an optimized routing system is needed to determine the shortest available delivery paths.																				
Why it is important	• Enables faster delivery of emergency supplies. • Reduces rescue response time. • Improves the utilization of limited emergency resources. • Supports decision-making during disaster situations. • Helps save lives by selecting the shortest available routes.																				
Problem specification	Input • Locations (hospital, road junctions, villages) • Road connections • Travel time for each road • Blocked roads Output • Shortest path from Bentong Hospital to each village • Total travel time • Notification if a village is inaccessible Objective: Minimize the total travel time while avoiding blocked roads																				
Potential solutions	<table><tr><td>Algorithm</td><td>Reason</td></tr><tr><td>Dynamic Programming</td><td>Can solve optimization problems but is more complex that required</td></tr><tr><td>Greedy Algorithm</td><td>Dijkstra's algorithm is based on the greedy paradigm</td></tr></table>			Algorithm	Reason	Dynamic Programming	Can solve optimization problems but is more complex that required	Greedy Algorithm	Dijkstra's algorithm is based on the greedy paradigm												
Algorithm	Reason																				
Dynamic Programming	Can solve optimization problems but is more complex that required																				
Greedy Algorithm	Dijkstra's algorithm is based on the greedy paradigm																				

	Graph Algorithm	Models road networks naturally and efficiently finds shortest paths
Sketch (framework, flow, interface)	 <pre> graph TD Start([Start]) --> Input1[/Input Road Network Data/] Input1 --> Input2[/Input Blocked Roads/] Input2 --> RunDijkstra[Run Dijkstra Algorithm] RunDijkstra --> Calculate[Calculate Shortest Routes] Calculate --> Display[/Display Route & Travel Time/] Display --> End([End]) </pre> The flowchart illustrates the process of finding shortest paths using a graph algorithm. It begins with a 'Start' terminal, followed by two input steps: 'Input Road Network Data' and 'Input Blocked Roads'. The core of the algorithm is 'Run Dijkstra Algorithm', which leads to 'Calculate Shortest Routes'. The final output step is 'Display Route & Travel Time', followed by an 'End' terminal.	

Project Proposal Refinement (week 11, submission date: 12 June 2026)

Group Name	Group 8													
Members	<table><tr><th>Name</th><th>Role</th></tr><tr><td>NURUL AINI BINTI ISMAIL</td><td>Scenario Lead</td></tr><tr><td>NURUL SYAZANA BINTI ZAIDI</td><td>Research Lead</td></tr><tr><td>NUR SOFEA BINTI MUHALIS</td><td>Algorithm Design Lead</td></tr><tr><td>ALYA MAISARAH HANNANI BINTI AHMAD SUKRI</td><td>Programming Lead</td></tr><tr><td>RASYIDAH LIANA BINTI ABD HALIM</td><td>Analysis & Portfolio Lead</td></tr></table>		Name	Role	NURUL AINI BINTI ISMAIL	Scenario Lead	NURUL SYAZANA BINTI ZAIDI	Research Lead	NUR SOFEA BINTI MUHALIS	Algorithm Design Lead	ALYA MAISARAH HANNANI BINTI AHMAD SUKRI	Programming Lead	RASYIDAH LIANA BINTI ABD HALIM	Analysis & Portfolio Lead
	Name	Role												
	NURUL AINI BINTI ISMAIL	Scenario Lead												
	NURUL SYAZANA BINTI ZAIDI	Research Lead												
	NUR SOFEA BINTI MUHALIS	Algorithm Design Lead												
	ALYA MAISARAH HANNANI BINTI AHMAD SUKRI	Programming Lead												
	RASYIDAH LIANA BINTI ABD HALIM	Analysis & Portfolio Lead												
Problem statement	Heavy rainfall has triggered landslides that block major roads connecting Bentong Hospital to nearby villages. Rescue teams need an efficient method to determine the shortest available routes for delivering emergency supplies while avoiding blocked roads.													
Objectives	• Model the transportation network as a weighted graph. • Compare algorithm paradigms. • Select the most suitable algorithm. • Implement the solution in Java. • Analyse algorithm correctness and time complexity.													
Expected output	• Shortest route to each village. • Total travel time. • Notification if a village is inaccessible. • Java program output.													
Problem scenario description	The scenario is based on a landslide disaster in Bentong, Pahang. Multiple villages become isolated due to road blockages caused by continuous rainfall. Bentong Hospital acts as the emergency operations centre responsible for coordinating relief supply deliveries.													
Why it is important	Efficient routing minimizes delays, reduces operational costs, optimizes resource allocation, and improves disaster response, ultimately helping to save lives.													
Problem specification	Input • Graph vertices (Hospital, Junctions, Villages) • Weighted edges (Travel time) • Blocked roads Process • Build a weighted graph. • Execute Diikstra's Algorithm.													

	• Compute shortest paths. Output • Shortest route • Travel time • Inaccessible villages
Potential solutions	After comparing Sorting, Divide and Conquer, Dynamic Programming, Greedy, and Graph algorithms, Dijkstra's Algorithm (Graph + Greedy paradigm) is selected because it efficiently computes the shortest paths in a weighted graph with non-negative edge weights.
Sketch (framework, flow, interface)	



	Scenario selection and project planning	wk10
	Literature review and algorithm comparison	wk10
	Algorithm design (graph model, pseudocode, flowchart)	wk10
	Java programming and debugging	wk11
	Program testing and validation	wk11
	Algorithm correctness and complexity analysis	wk12
	Online portfolio development	wk12
	Presentation preparation	wk13

Project Progress (Week 10 – Week 14)

Milestone 1	Problem identification and Algorithm Selection														
Date (week)	Week 10-11														
Description / sketch	- Selected the landslide emergency supply delivery scenario. - Defined the problem statement and objectives. - Comparing algorithm paradigms. - Choose Dijkstra's Algorithm. - Designed the initial graph structure.														
Role	<table><tr><td>Nurul Aini</td><td>Nurul Syazana</td><td>Nur Sofea</td><td>Alya Maisarah</td><td>Rasyidah Liana</td></tr><tr><td>Scenario development</td><td>Algorithm Research</td><td>Graph model & Pseudocode</td><td>Initial Java program</td><td>Complexity research and portfolio planning</td></tr></table>					Nurul Aini	Nurul Syazana	Nur Sofea	Alya Maisarah	Rasyidah Liana	Scenario development	Algorithm Research	Graph model & Pseudocode	Initial Java program	Complexity research and portfolio planning
Nurul Aini	Nurul Syazana	Nur Sofea	Alya Maisarah	Rasyidah Liana											
Scenario development	Algorithm Research	Graph model & Pseudocode	Initial Java program	Complexity research and portfolio planning											

Milestone 2	Algorithm Implementation, Testing and Documentation														
Date (Wk)	Week 12-Week 14														
Description / sketch	- Completed Java implementation. - Conducted testing using sample data. - Analysed correctness and time complexity. - Developed the online portfolio. - Prepared presentation slides and final report.														
Role	<table><tr><td>Nurul Aini</td><td>Nurul Syazana</td><td>Nur Sofea</td><td>Alya Maisarah</td><td>Rasyidah Liana</td></tr><tr><td>Finalize Scenario & report</td><td>Finalize comparison section</td><td>Complete algorithm design</td><td>Complete code & testing</td><td>Complexity analysis, portfolio</td></tr></table>					Nurul Aini	Nurul Syazana	Nur Sofea	Alya Maisarah	Rasyidah Liana	Finalize Scenario & report	Finalize comparison section	Complete algorithm design	Complete code & testing	Complexity analysis, portfolio
Nurul Aini	Nurul Syazana	Nur Sofea	Alya Maisarah	Rasyidah Liana											
Finalize Scenario & report	Finalize comparison section	Complete algorithm design	Complete code & testing	Complexity analysis, portfolio											